

Req ID	Instruction / Feature	Verification Requirement	Corner Cases / Stimulus	Expected Result / Checks	
IF-001	Instruction Fetch	Verify instruction fetch through OBI IF	grant immediate/delayed, rvalid immediate/delayed	Correct instr_addr_o, request/handshake behavior, correct instruction consumed	
IF-002	Pipeline	Verify pipeline survives IF backpressure	random instr_gnt_i/instr_rvalid_i latency	No instruction lost, duplicated, or executed incorrectly	
IF-003	x0	Verify architectural x0 remains zero	Use x0 as rs1/rs2 and rd for all applicable instructions	Reads return 0; writes to x0 have no architectural effect	
ALU-001	ADD	Verify 32-bit addition	0+0, MAX+1, 0xFFFFFFFF+1, positive/negative operands, overflow	rd=(rs1+rs2) mod 2^32; no trap/data OBI	
ALU-002	SUB	Verify 32-bit subtraction	equal operands, 0-1, MIN/MAX, underflow	rd=(rs1-rs2) mod 2^32	
ALU-003	SLL	Verify logical left shift	shift amounts 0,1,31, rs2=32/33/63/FFFFFFFF	Shift amount=rs2[4:0]; zero fill	
ALU-004	SLT	Verify signed less-than	equal, -1 vs 0, INT_MIN vs INT_MAX	rd=1 iff signed(rs1)<signed(rs2)	
ALU-005	SLTU	Verify unsigned less-than	0, FFFFFFFF, signed-looking operands	rd=1 iff unsigned(rs1)<unsigned(rs2)	
ALU-006	XOR	Verify bitwise XOR	all-0, all-1, alternating patterns, same operands	Correct 32-bit XOR	
ALU-007	SRL	Verify logical right shift	shamt 0/1/31/>31	rs2[4:0] used; zero fill	
ALU-008	SRA	Verify arithmetic right shift	positive/negative, shamt 0/1/31/>31	Sign bit replicated	
ALU-009	OR	Verify bitwise OR	zeros, ones, complementary patterns	Correct OR result	
ALU-010	AND	Verify bitwise AND	zeros, ones, complementary patterns	Correct AND result	
ALUI-001	ADDI	Verify add-immediate	imm=0, 1, -1, +2047, -2048, arithmetic wraparound	Correct sign-extension and 32-bit result	
ALUI-002	SLTI	Verify signed immediate comparison	immediate boundaries -2048/+2047; signed extrema	Correct signed comparison	
ALUI-003	SLTIU	Verify unsigned comparison with sign-extended immediate	imm=-1,0,1,+2047	Immediate first sign-extended to XLEN, then unsigned comparison	
ALUI-004	XORI	Verify immediate XOR	imm 0, -1, boundaries	Correct sign-extension/XOR	
ALUI-005	ORI	Verify immediate OR	imm 0, -1, boundaries	Correct result	
ALUI-006	ANDI	Verify immediate AND	imm 0, -1, boundaries	Correct result	
ALUI-007	SLLI	Verify immediate left shift	shamt=0,1,31	Correct shift using legal RV32 shamt	
ALUI-008	SRLI	Verify immediate logical right shift	shamt=0,1,31	Zero-filled result	
ALUI-009	SRAI	Verify immediate arithmetic right shift	shamt=0,1,31; ± operands	Correct sign extension	
U-001	LUI	Verify upper immediate	imm=0, 0xFFFF, bit19 set/unset	rd=imm20<<12	
U-002	AUIPC	Verify PC-relative upper immediate addition	PC boundary values, imm=0/max/sign bit set	rd=PC+(imm20<<12) modulo 2^32	
JMP-001	JAL	Verify direct jump and link	±offset, forward/backward, rd=x0, target boundaries	rd=instruction PC+4; next architectural PC=PC+offset	
JMP-002	JALR	Verify indirect jump/link	imm=0, ±2048 range, rd=x0, rs1=x0	rd=PC+4; target=(rs1+signext(imm)) & ~1	
JMP-003	JALR	Verify JALR bit-0 clearing	computed target odd	bit 0 cleared according to ISA	
BR-001	BEQ	Verify equal branch	equal/not equal, zero/extreme values	Branch iff rs1==rs2	
BR-002	BNE	Verify not-equal branch	equal/not equal	Branch iff rs1!=rs2	
BR-003	BLT	Verify signed less-than branch	-1 vs 0, MIN/MAX, equal	Correct signed decision	
BR-004	BGE	Verify signed >= branch	signed extrema, equal	Correct signed decision	
BR-005	BLTU	Verify unsigned less-than	0/FFFFFFFF, equal	Correct unsigned decision	
BR-006	BGEU	Verify unsigned >=	0/FFFFFFFF, equal	Correct unsigned decision	
BR-007	All branches	Verify taken/not-taken control flow	zero offset, min/max legal offsets, forward/backward	Correct target; wrong-path instruction has no architectural side effect	
LD-001	LB	Verify signed byte load	byte lanes 0..3, data 00/7F/80/FF	Selected byte sign-extended to 32 bits	
LD-002	LBU	Verify unsigned byte load	all byte lanes, 00/7F/80/FF	Selected byte zero-extended	
LD-003	LH	Verify signed halfword load	legal lane offsets 0/2; 0000/7FFF/8000/FFFF	Correct lane selected/sign-extended	
LD-004	LHU	Verify unsigned halfword load	offsets 0/2; boundary values	Correct zero extension	
LD-005	LW	Verify word load	0, all-1, patterns, aligned addresses	Exact 32-bit memory value written to rd	
LD-006	All loads	Verify OBI read handshake	delayed grant, delayed rvalid, varying latency	Correct request/address; WB only from correct response; pipeline handles wait	
ST-001	SB	Verify byte store	address offsets 0,1,2,3; varied data	Correct address/byte-enable/write-data lane behavior	
ST-002	SH	Verify halfword store	aligned offsets 0,2	Correct BE and appropriate write-data lanes	
ST-003	SW	Verify word store	aligned addresses; varied data	data_be_o=1111; correct data/address	
ST-004	All stores	Verify OBI write handshake	delayed grant/backpressure	Request stable/handled per OBI protocol; exactly one architectural store	
MEM-001	Address generation	Verify rs1 + signext(imm)	±offset, address wraparound	Correct effective address modulo 2^32	
MUL-001	MUL	Verify low 32-bit multiply	0,1,-1, MIN/MAX, overflow, random	low 32 bits of 64-bit product	
MUL-002	MULH	Verify signed×signed high result	± operands, MIN/MAX	bits 63:32 of signed 64-bit product	
MUL-003	MULHSU	Verify signed×unsigned high result	negative rs1; large unsigned rs2; extrema	Correct high 32 bits of signed×unsigned product	
MUL-004	MULHU	Verify unsigned×unsigned high result	0/1/FFFFFFFF/boundaries	Correct upper product	
MUL-005	Multiply pipeline	Verify multi-cycle operation behavior	dependencies before/after MUL, memory/branch adjacent	No premature WB; correct final result; no duplicated/skipped instruction	
DIV-001	DIV	Verify signed division	± operands, exact/non-exact quotient, extrema	Signed quotient truncated toward zero	
DIV-002	DIVU	Verify unsigned division	0,1,MAX and random operands	Correct unsigned quotient	
DIV-003	REM	Verify signed remainder	± operands, exact/non-exact	rem=dividend-quotient*divisor, sign follows dividend	
DIV-004	REMU	Verify unsigned remainder	boundaries/random	Correct unsigned remainder	
DIV-005	DIV by zero	Verify divide-by-zero ISA result	divisor=0; varied dividend	DIV/DIVU rd=0xFFFFFFFF	

DIV-006	REM by zero	Verify remainder-by-zero result	divisor=0	REM/REMU rd=dividend	
DIV-007	Signed overflow	Verify INT_MIN / -1	80000000 / FFFFFFFF	DIV=80000000; REM=00000000	
DIV-008	Divide pipeline	Verify long-latency behavior	data dependencies, preceding/following ops	Correct stall/interlock and single WB; don't require arbitrary fixed cycle count unless spec mandates it	
EXC-001	Misaligned LW	Verify word misalignment handling	EA mod4 =1,2,3	Behavior matches configured CV32E40P architectural/spec behavior; no incorrect architectural load	
EXC-002	Misaligned SW	Verify word misalignment handling	EA mod4=1,2,3	Spec-correct LSU/trap behavior; no unintended memory corruption	
EXC-003	Misaligned LH/LHU	Verify halfword misalignment	odd EA	Spec-correct handling	
EXC-004	Misaligned SH	Verify halfword misalignment	odd EA	Spec-correct handling	
EXC-005	Byte access	Verify byte accesses are never alignment-faulted	all address low bits	SB/LB/LBU operate at every byte offset	
EXC-006	Instruction-address misalignment	Verify taken control transfer to invalid alignment	JAL/branch target with bit1=1; applicable JALR target	Instruction-address-misaligned behavior per RV32I/core configuration	
EXC-007	Trap redirection	Verify applicable exception redirect	trap-vector input varied	Fetch redirects to configured trap vector; faulting op has no unintended architectural result	
HZD-001	RAW dependencies	Verify back-to-back dependencies	producer rd used immediately as rs1/rs2	Correct forwarding/stall behavior	
HZD-002	Load-use dependency	Load followed by consumer	zero/random OBI latency	Consumer receives loaded value	
HZD-003	Long-latency dependency	MUL/DIV followed by dependent op	rd→rs1/rs2	Consumer executes only with correct result	
HZD-004	Control hazard	Taken jump/branch followed by side-effect instruction	store/register write on wrong path	Wrong-path side effects suppressed	
REG-001	rd=x0	Run every rd-writing instruction with rd=x0	ALU/load/MUL/DIV/jumps	x0 remains zero	
REG-002	Register aliasing	Verify rdrs1, rdrs2, rs1==rs2	all applicable instructions	Sources use pre-instruction values; final rd correct	